MyCodeSchool

Code review — python-initiere

Runde de review pe proiectul tău, cea mai nouă sus.

Prima ta rundă în Python: `functii.py` + `main.py`, verificate prin rulare.

Runda 1 — 2026-10-27 — sha 6158545

Stare: `python main.py` se oprește înainte să ruleze ceva — `IndentationError` în `functii.py`. B2–B4 au fost reproduse separat, pe copii ale funcțiilor; B5 e pe codul tău, după ce B1–B3 sunt reparate.

🔴 Critice

B1 🔴 Corpul lui `while` nu e indentat — `functii.py:9–10`

CUM E ACUM: `IndentationError: expected an indented block after 'while' statement on line 8`

DE CE: în C++ blocul e dat de acolade; indentarea e doar pentru ochi. În Python e invers: **indentarea ESTE blocul**. După `:` (la `def`, `if`, `while`, `for`) interpretorul așteaptă cel puțin o linie împinsă mai la dreapta. `n=n/10` stă la aceeași coloană cu `while`, deci pentru Python bucla n-are corp — și refuză să citească tot fișierul, nu doar linia aceea. De-aia nici `ultima_cifra`, care e corectă, nu poate fi folosită.

FIX: 4 spații în plus pe liniile 9 și 10.

B2 🔴 `nr++` nu există în Python — `functii.py:10`

CUM E ACUM: `SyntaxError: invalid syntax`

DE CE: Python n-are operatorii `++` / `--`. Un `int` în Python nu e o căsuță din memorie pe care o modifici pe loc, ci un obiect care nu se schimbă; `nr += 1` construiește valoarea nouă și leagă numele `nr` de ea. Un `++` care „crește variabila pe loc" n-ar avea ce să crească.

FIX: `nr += 1`.

B3 🔴 `/` dă număr real, nu întreg — `functii.py:9`

CUM E ACUM: `numar_cifre(1234)` → **327**, nu 4 (rulat pe funcția ta cu B1 și B2 reparate).

DE CE: în C++ `n/10` cu `n` de tip `int` e împărțire întreagă, pentru că tipul variabilei decide. În Python `/` dă **mereu** `float`: `1234/10` = `123.4`, apoi `12.34`, `1.234`, `0.1234`... `n` nu ajunge niciodată exact la 0 prin tăiere de cifre — ajunge doar când numărul devine atât de mic încât nu mai încape într-un `float`, după 327 de pași. Împărțirea întreagă din C++ are în Python operatorul ei: `//`.

FIX: `n = n // 10`.

B4 🔴 **Apelezi o funcție care nu există** — `main.py:9`

CUM E ACUM: `NameError: name 'numar_cifra' is not defined. Did you mean: 'numar_cifre'?`

DE CE: în `import` ai `numar_cifre`, în apel `numar_cifra`. În C++ asta ar fi oprit compilarea; Python nu compilează înainte, așa că greșeala apare abia când linia chiar se execută. Citește ultima linie din mesaj: Python îți spune singur ce nume a găsit.

FIX: `print(numar_cifre(1234))`.

B5 🔴 **`numar_cifre(0)` întoarce 0** — `functii.py:6–11`

CUM E ACUM: după B1–B3 reparate, `numar_cifre(0)` → **0**. Numărul 0 are o cifră.

DE CE: bucla `while n != 0` nu intră deloc când `n` pornește de la 0, deci `nr` rămâne 0. E același caz-limită ca în C++: o buclă care numără „cât timp mai sunt cifre" trebuie să facă măcar un pas.

FIX: pornești cu `nr = 1` și numeri cât timp `n >= 10` — sau tratezi 0 separat, înainte de buclă.

🟡 Importante

M1 🟡 **`__pycache__/` e comis** — `__pycache__/functii.cpython-315.pyc` Python îl generează singur la fiecare rulare, ca `obj/` din Code::Blocks. Adaugă `.gitignore` cu `__pycache__/` și `.venv/`, apoi `git rm -r --cached __pycache__`.

M2 🟡 **VS Code folosește Python-ul de sistem** — `.vscode/settings.json:2` `"ms-python.python:system"` → proiectul nu rulează din `.venv`. Ctrl+Shift+P → *Python: Select Interpreter* → `.venv`.

🟢 Cleanups

C1 🟢 **Spații în jurul operatorilor** — `functii.py`, `main.py` `ultima=n%10` → `ultima = n % 10`; `def ultima_cifra(n: int) -> int:`. În Python stilul e standard (PEP 8), nu la alegere.

C2 🟢 **Paranteze inutile la `while`** — `functii.py:8` `while(n!=0):` → `while n != 0:`. Merge și cu paranteze, dar în Python nu se pun.

Before / After

	Acum	Corect

B1 + B2 + B3	`while(n!=0):`
	`n=n/10`
	`nr++`
		`while n != 0:`
		`    n = n // 10`
		`    nr += 1`

B4	`print(numar_cifra(1234))`	`print(numar_cifre(1234))`
B5	`nr=0` `while(n!=0):`	`nr = 1` `while n >= 10:`

Q&A

1. Ce afișează `print(7 / 2, 7 // 2)` în Python? Dar în C++, `cout << 7/2;` cu `int`? De ce diferă?
2. Șterge un spațiu din fața unei linii din interiorul unui `if`. Ce eroare primești și pe ce linie?
3. Ce întoarce funcția ta reparată pentru `numar_cifre(5)`, `numar_cifre(10)` și `numar_cifre(0)`? Verifică-le cu `print` în `main.py`.